

Module 8: Anchors

Instructional Hours: 6

Module Overview

In this module, you will learn about anchors. This is a technique that is used to explain a single instance by developing if then rules. The output of this technique is text based. You will review some study materials on anchors and get to implement the technique using Python

Learning Outcomes

At the end of this module, you will be able to:

1. Explain the principles behind use of Anchors in Making AI models interpretable
2. Describe The Process of Generation of Anchors for an AI model
3. Create code to implement Anchors in Python to show how a data instances was predicted
4. Interpret anchor rules to stakeholders

Learning Activities/task List

1. Review Introductory notes on Anchors
2. Contribute to a discussion on Anchors
3. Review selected readings and examples on the Anchors
4. Complete a Quiz on Anchors
5. Implement Python code to generate explanations using Anchors
6. Complete a Final module quiz

Introduction to Anchors

Anchors are a local model-agnostic explainable AI technique in which if then rules are generated to explain model predictions. These rules are derived from specific feature values in the input data, which strongly influence the AI's decision. Anchors work by identifying "anchors" which are subsets of feature values that, when fixed, guarantee the same prediction within a particular range of similar inputs. This gives users a clear and straightforward explanation of why the model made a particular decision.

How Anchors Work

Anchors rely on the following principles to provide explanations:

Identifying Important Features

For a given input, the anchors algorithm first examines the feature values that contributed most significantly to the prediction. This is determined by fixing the values of such features and varying the other features. If the accuracy (precision) of the model remains high (predicts most of the varied instances in the same way as the original sentence) in spite of the variance in the other features, then these features are important in the prediction of the instance in consideration. These features values are potential candidates for forming the anchor.

As an example suppose we have the sentence "The show is not good" for which we have a model that performs sentiment classification and it classifies this sentence as negative. If a potential anchor "not, good" (i.e. contains the words not and good) and when we vary (perturb) other words in the sentence the model still predicts negative, then we can conclude that the feature "not, good" is very important for the prediction. As an example, we can vary the sentence above to "it was not a good performance". This varied sentence still contains the words "not" and "good". The anchors precision will be increased if the black-box model predicts this sentence as negative. The more perturbed sentences it predicts correctly, the higher it's precision.

Any potential anchor will be compared with other potential anchor to pick the most precise. As an example, another potential anchor for the above sentence is "is, not". Another is "was, a, good". Anchors that are shorter in length (consider less features given the same precision) are preferred.

Constructing If-Then Rules

Anchors generate rules that resemble human reasoning. For example, in a model predicting whether a customer will default on a loan, an Anchor might state: "If the customer's credit score is above 700 and income is over \$50,000, the model predicts no default with high confidence."

Coverage

Coverage refers to the number of instances in the perturbed dataset for which the rule applies. This is determined when trying to add a new feature. As an example if we have the anchor {not}, we can extend it to {not, good} or {not, was}. If these have equal precision, then we will prefer the rule that matches more instances from the sample dataset that matched the original {not} anchor.

If we find that {not, good} is more precise, and it's precision is equal to the {not} anchor but the coverage of the {not} is greater, then we will prefer the

shorter rule.

Detailed Review of Anchors

Reading Material

To learn more about Anchors, read the following materials

- **Scoped Rules (Anchors)**

Molnar, C. (2022). Interpretable Machine Learning. A Guide for Making Black Box Models Explainable (2nd ed.),

Implementation of Anchors Using Python

You can implement the Anchors using python. In this exercise, you will generate explanations for a random forest classification of instances in the iris dataset

```
#Install necessary libraries
!pip install alibi
#import the necessary packages
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

from alibi.explainers import AnchorTabular
from sklearn.metrics import accuracy_score
# Load the iris dataset
data = load_iris()
X, Y = data.data, data.target
feature_names = data.feature_names
class_names = data.target_names
# Split the data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(X, Y,
test_size=0.2, random_state=42)
#Put the data in a dataframe for easier visualization
columns=['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
import pandas as pd
df = pd.DataFrame(x_test, columns=columns)
df['target'] = y_test
df.head(2)
# Train a RandomForestClassifier
model = RandomForestClassifier(n_estimators=50, random_state=42)
```

```

model.fit(x_train, y_train)
predictions =model.predict(x_test)
accuracy=accuracy_score(y_test, predictions)
print("Accuracy: {:.2f}%".format(accuracy*100))
#print the targets
targets=list(data.target_names)
print(targets)
#print the prediction of the first item
pred=model.predict(x_test[0].reshape(1,-1))
print("Actual {} ; Predicted {}".format(targets[y_test[0]],
targets[pred[0]]))
# Initialize the Anchor explainer
explainer = AnchorTabular(model.predict, feature_names=feature_names,
categorical_names={})
# Fit the explainer to the data
explainer.fit(x_test, disc_perc=(20,40, 60, 60)) # Discretize
continuous variables for explanation

# Pick a sample from the test set to explain
sample = x_test[1].reshape(1, -1)
print(f"Model prediction: {class_names[model.predict(sample)[0]]}")
#Explain the model's prediction for the selected instance
explanation = explainer.explain(sample, threshold=0.95)
#Print the anchor explanation
print(f'Anchor: {" AND ".join(explanation.anchor)}')
print(f'Precision: {explanation.precision}')
print(f'Coverage: {explanation.coverage*100}%')
#view the sample
df.iloc[1].head()

```